

# **Week 2 Lesson: Introduction to Python and Computing Environments**

**23<sup>rd</sup> September 2026**

Dr. Joan Onyango, Lecturer, JKUAT

## **Lesson content:**

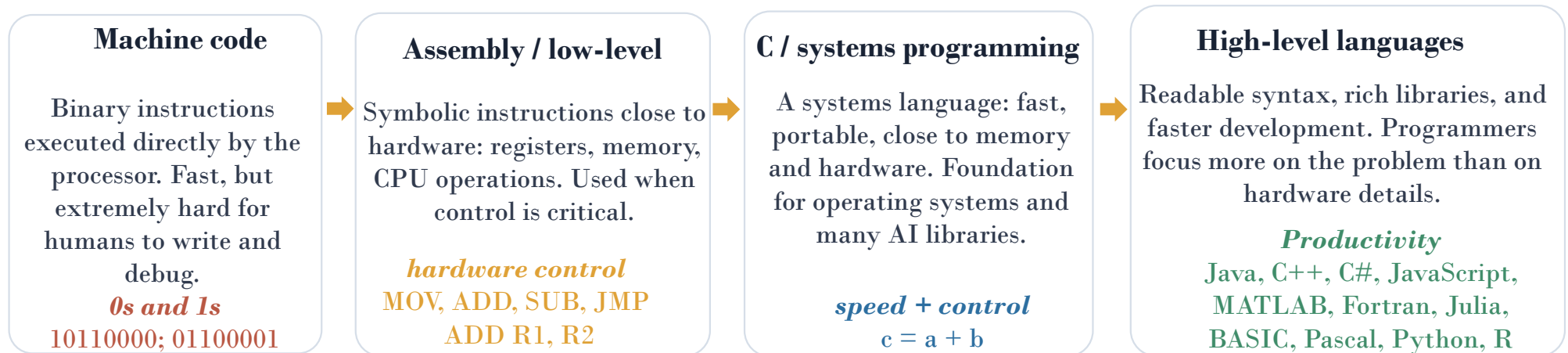
- Introduction to Python for Scientific Computing and AI/ML
- Setting Up a Python Computing Environment
- Integrated Development Environments and Notebook Environments
- Python Fundamentals
- Python Libraries for Scientific Computing and Machine Learning
- Working with Data in Python
- Git and GitHub for Reproducible Workflows

# Language of Communicating with Machines

Programming languages translate human ideas into instructions that a computer can execute.

**Human problem → Algorithm → Programming language →  
Compiler / Interpreter → Machine instructions → Output**

**Abstraction ladder: from machine-close control to human-friendly productivity**



**Hardware control → easier programming → greater programmer productivity**

## What is Python?

A high-level programming language that we use to communicate our instructions to the computer.

## Why Python?

- ✓ Easy-to-read syntax
- ✓ Widely used in AI, ML, data science, engineering, and research

## How do we execute Python?

| Option                                                                                                     | What It Is                                                             | How It Works                                                        |
|------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|---------------------------------------------------------------------|
|  <b>Google Collab</b>      | Cloud-based Python environment                                         | Write and run Python in a web browser                               |
|  <b>Jupyter Notebook</b> | Interactive notebook environment                                       | Run Python code in individual cells and view outputs interactively  |
|  <b>Anaconda</b>         | Python distribution and package manager                                | Installs Python plus commonly used data science libraries and tools |
|  <b>Python + IDE</b>     | Python installed on your computer and used with an IDE such as VS Code | Write .py files and run them locally                                |

## Jupyter vs Google Collab

|                            | Jupyter Notebook                             | Google Colab                                               |
|----------------------------|----------------------------------------------|------------------------------------------------------------|
| <b>Where it runs</b>       | Usually on your own <b>computer</b>          | In the cloud, through a <b>web browser</b>                 |
| <b>Installation</b>        | Usually requires Python/Jupyter installation | No installation required                                   |
| <b>Internet</b>            | Can work <b>offline</b>                      | Requires <b>internet</b> connection                        |
| <b>Computing resources</b> | Uses your computer's resources               | Uses Google's cloud resources                              |
| <b>Sharing</b>             | You share the notebook/file                  | Easy to share using a Google link                          |
| <b>Collaboration</b>       | More limited                                 | Easier for collaborative work                              |
| <b>Libraries</b>           | You install them on your computer            | Many common ML/data science libraries are pre-installed    |
| <b>Best for</b>            | Local research and development               | Teaching, workshops, quick experiments, and cloud-based ML |

## Starting Google Collab

- Go to your browser
- Sign in to your Google account
- In a new tab, open Google Collab

```
 print("Hello, Colab!")
```

# Google Collab Anatomy

## **Notebook**

→ Our workspace

## **Code Cell**

→ Where we write Python instructions

## **Text/Markdown Cell**

→ Where we write headings, explanations, equations, and notes

## **Run**

→ Executes the selected code

## **Output**

→ View the results, tables, charts, errors, and files

# Google Collab Anatomy

co

Untitled7.ipynb

☆

File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all

⋮

📄

<>

🔍

🔑

📁

📊

[ ]

▶ Start coding or generate with AI.

Gemini

How can I help you today?

How can I install Python libraries?

Load data from Google Drive

Show an example of training a simple ML model

What can I help you build?

+ 📄

Gemini 2.5 Flash ▶

🔗 Variables

📄 Terminal

## **Google Collab Essential habits**

### **Name your notebook clearly**

Example: intro\_python\_colab\_lesson1.ipynb

### **Check autosave to your Google Drive**

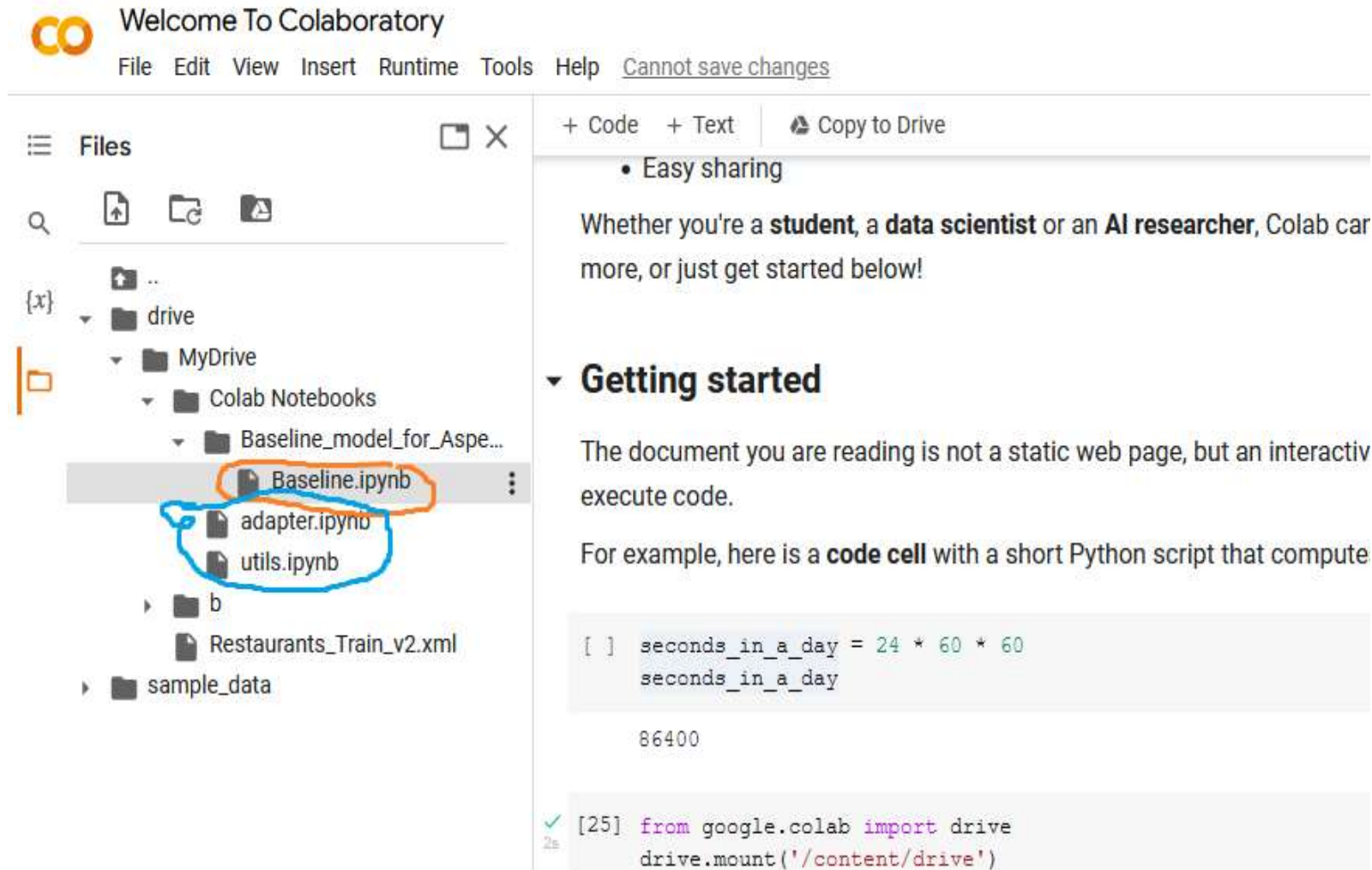
Use File → Save a copy in Drive when needed.

### **Choose runtime / compute**

Runtime → Change runtime type → CPU, GPU, or TPU.



# Navigate the Google Collab Notebook Interface



Welcome To Colaboratory

File Edit View Insert Runtime Tools Help [Cannot save changes](#)

Files

drive

MyDrive

Colab Notebooks

Baseline\_model\_for\_Aspe...

Baseline.ipynb

adapter.ipynb

utils.ipynb

b

Restaurants\_Train\_v2.xml

sample\_data

+ Code + Text Copy to Drive

• Easy sharing

Whether you're a **student**, a **data scientist** or an **AI researcher**, Colab can more, or just get started below!

▼ Getting started

The document you are reading is not a static web page, but an interactive execute code.

For example, here is a **code cell** with a short Python script that compute

```
[ ] seconds_in_a_day = 24 * 60 * 60
seconds_in_a_day

86400
```

```
✓ [25] from google.colab import drive
2s drive.mount('/content/drive')
```

1 **Top menus:** File, Edit, View, Insert, Runtime, Tools, Help

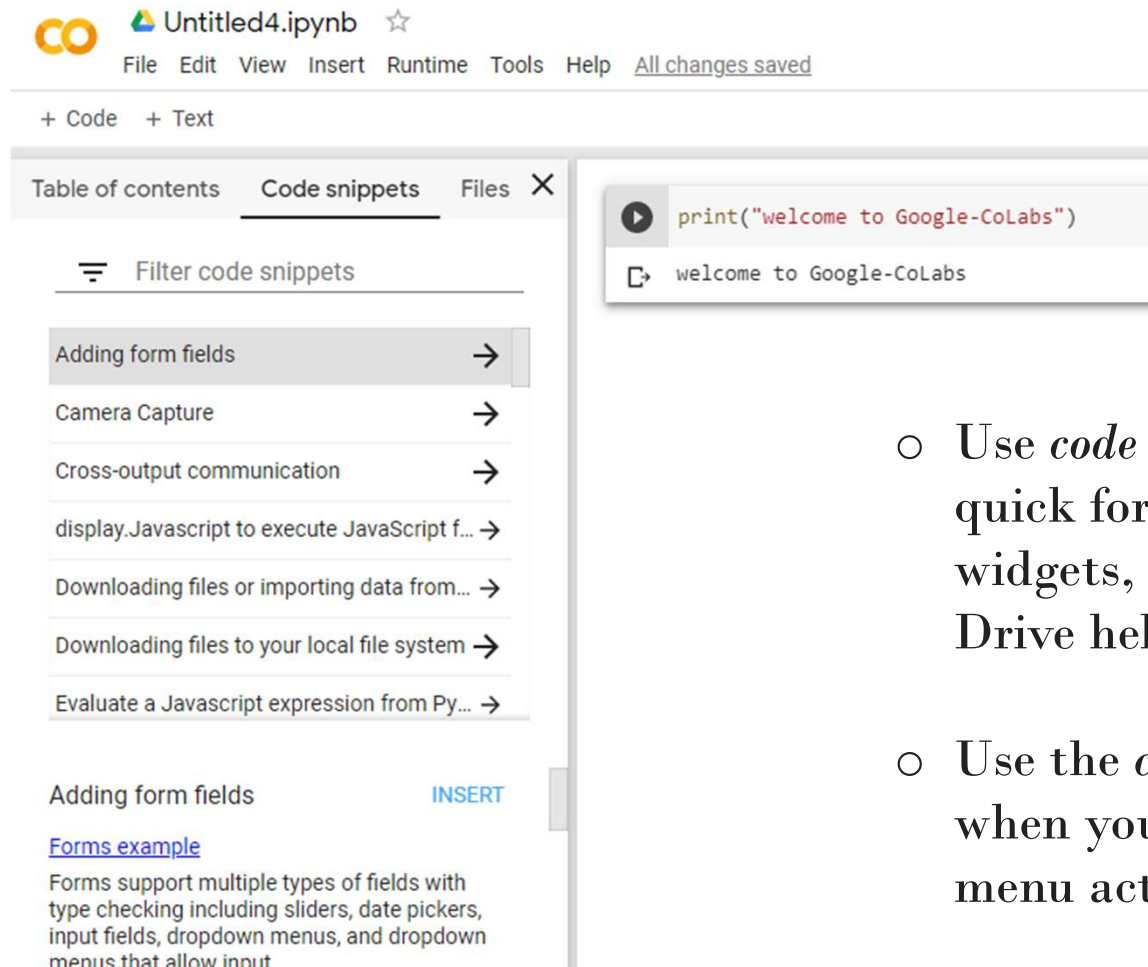
2 **Notebook cells:** code, text, outputs, errors

3 **Left sidebar:** TOC, search, variables, files, secrets

Colab notebook with left Files panel

# Code Snippets and Command Palette

Colab has discoverable examples and searchable commands.

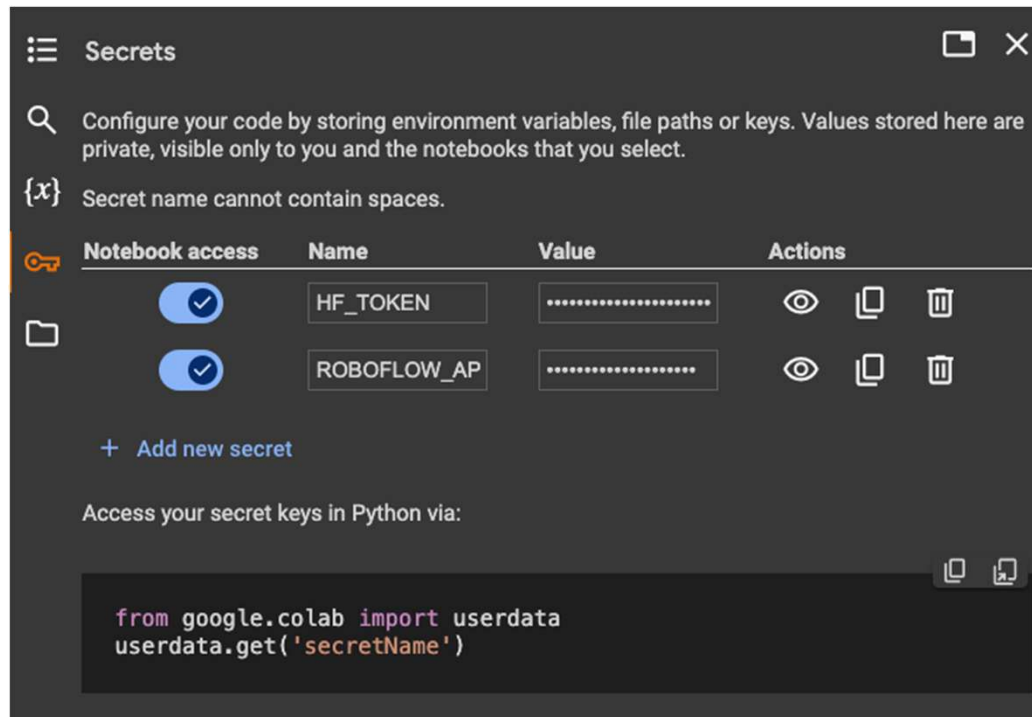


Code snippets panel

- Use *code snippets* for quick forms, charts, widgets, and Google Drive helpers.
- Use the *command palette* when you forget where a menu action lives.

# Secrets: safe API keys and tokens

Do not paste private keys directly into cells before sharing a notebook.

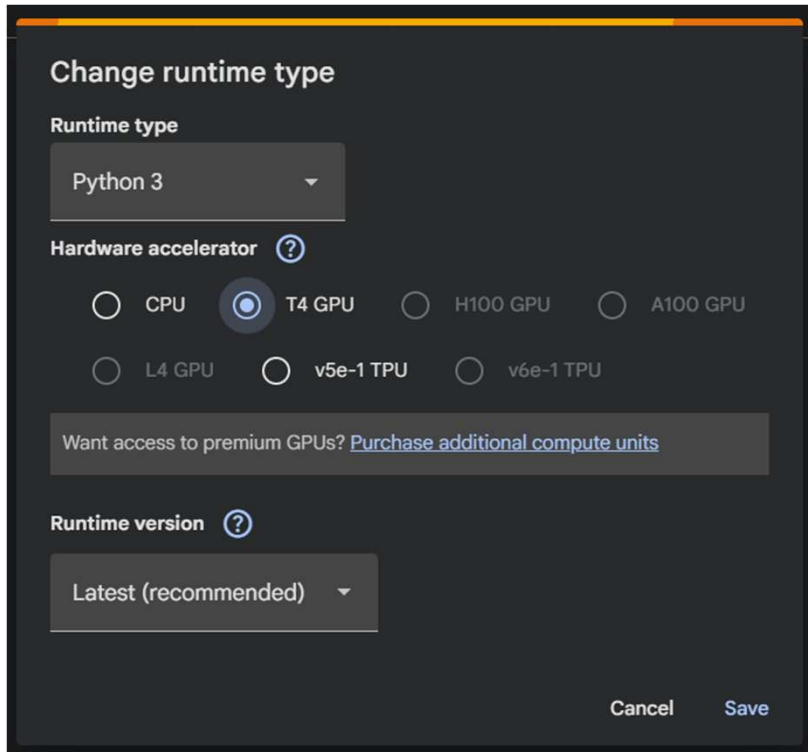


Secrets panel: masked values  
and notebook access

- From Google Colab import userdata
- GEMINI\_API\_KEY =  
    userdata.get('GEMINI\_API\_KEY')
- HF\_TOKEN = userdata.get('HF\_TOKEN')
- # Use token in libraries without printing it
- ✓ Create secret → give notebook access → read it with userdata.get().
- ✓ Never print secrets; never store them in GitHub notebooks.
- ✓ Use separate development and production keys when possible.

# Runtimes and accelerators

Choose the compute environment before long training jobs.



The screenshot shows a dark-themed dialog box titled "Change runtime type". It contains the following elements:

- Runtime type:** A dropdown menu currently showing "Python 3".
- Hardware accelerator:** A section with a help icon (?) and six radio button options: CPU, T4 GPU (which is selected), H100 GPU, A100 GPU, L4 GPU, v5e-1 TPU, and v6e-1 TPU.
- Premium GPUs:** A text box stating "Want access to premium GPUs? [Purchase additional compute units](#)".
- Runtime version:** A dropdown menu currently showing "Latest (recommended)" with a help icon (?) next to the label.
- Buttons:** "Cancel" and "Save" buttons at the bottom right.

- **CPU:** normal Python, data cleaning, small ML.
- **GPU:** deep learning, RAPIDS/cuDF, large tensor operations.
- **TPU:** selected TensorFlow/JAX workloads; setup differs from GPU.
- Free resources are not guaranteed; save checkpoints frequently.

## What Colab gives you

### **Zero setup**

Open a notebook and run Python without installing Anaconda, CUDA, or IDEs.

### **Compute on demand**

Use CPU, GPU, or TPU runtimes when available; resources are dynamic.

### **Shareable notebooks**

Save in Drive/GitHub and share like Google Docs.

### **AI assistance**

Use Gemini features to generate, explain, debug, and transform code.

## Python Programming Language : Practice

```
print("Hello, World!")
```

```
print("Welcome to Machine Learning!")
```

```
name = "Joan"  
print("Hello", name)
```

Python as a calculator:

```
2 + 3
```

```
10 * 5
```

```
100 / 4
```

```
x = 10
```

```
y = 5
```

```
x + y
```

# Variables and Data Types

## Variables

```
name = "Joan"
age = 25
height = 1.65
is_student = True
```

## Data type

| Data Type      | Example |
|----------------|---------|
| String (str)   | "Joan"  |
| Integer (int)  | 25      |
| Float (float)  | 1.65    |
| Boolean (bool) | True    |

## String

```
title = "This is a great book"
print(title)
print(title.lower())
print(title.upper())
len(title)
"great" in title
```

## List

```
ratings = [5, 4, 3, 5, 2]

print(ratings)
print(ratings[0])
```

## Conditions

```
rating = 5

if rating >= 4:
    print("Positive review")
else:
    print("Not a positive review")
```

## Loops

```
ratings = [5, 4, 3, 5, 2]

for rating in ratings:
    print(rating)
```

## Functions

```
def greet(name):
    print("Hello", name)

greet("Joan")
```

## Python for Data Science-*Libraries*

| Library              | Purpose                          |
|----------------------|----------------------------------|
| NumPy                | Numerical computing              |
| Pandas               | Working with tables and datasets |
| Matplotlib           | Data visualization               |
| Scikit-learn         | Building machine learning models |
| TensorFlow / PyTorch | Deep learning                    |

```
import pandas as pd

data = {
    "review": ["Great book", "Not useful", "Excellent"],
    "rating": [5, 2, 5]
}

df = pd.DataFrame(data)

df.head()          #df.head()
df.shape           #df.shape
df.columns         #df.columns
df["review"]       #df["review"]
df["rating"]       #df["rating"]
df.info()          #df.info()
```



# Working With a Dataset

Introduction\_to\_Python.ipynb

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

Step One: Mount Google Drive

```
[ ] from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
[ ] import pandas as pd
df = pd.read_csv('/content/drive/My Drive/AI for Science and Engineering Short Course/Sentiment Analysis/AmazonReviews.csv')
```

```
[ ] print('The number of reviews: ', len(df))
print(df[['title', 'rating']].head(10))
```

The number of reviews: 338

|   | title                                             | rating             |
|---|---------------------------------------------------|--------------------|
| 0 | Recommend getting, but also get proof of Pytho... | 5.0 out of 5 stars |
| 1 | Easy to Follow, Good Intro for Self Learner       | 5.0 out of 5 stars |
| 2 | Great for starting your journey on learning py... | 5.0 out of 5 stars |
| 3 | Great inner content! Not that great outer qual... | 4.0 out of 5 stars |
| 4 | Tips for beginners                                | 5.0 out of 5 stars |
| 5 | Great                                             | 5.0 out of 5 stars |
| 6 | Can be used both as a reference and a teaching... | 5.0 out of 5 stars |
| 7 | I have a whole bookshelf on Python books. Hand... | 5.0 out of 5 stars |
| 8 | simple y con el detalle de aprendizaje adecuado   | 5.0 out of 5 stars |
| 9 | Very detailed and clearly explained               | 5.0 out of 5 stars |

Gemini

Hello, Joan Atieno

How can I help you today?

How can I install Python libraries?

Load data from Google Drive

Show an example of training a simple ML model

What can I help you build?

Gemini 2.5 Flash

1:39 AM

# Working With a Dataset

 Introduction\_to\_Python.ipynb ☆ ☁

File Edit View Insert Runtime Tools Help

🔍 Commands + Code ▾ + Text | ▶ Run all ▾



### Step One: Mount Google Drive

[ ] ▶ from google.colab import drive  
drive.mount('/content/drive')

⋮ Mounted at /content/drive

[ ] import pandas as pd  
df = pd.read\_csv('/content/drive/My Drive/AI for Science and Engineering Short Course/Sentiment Analysis/AmazonReviews.csv')

[ ] print('The number of reviews: ', len(df))  
print(df[['title', 'rating']].head(10))

⌵ The number of reviews: 338

|   | title                                             | rating             |
|---|---------------------------------------------------|--------------------|
| 0 | Recommend getting, but also get proof of Pytho... | 5.0 out of 5 stars |
| 1 | Easy to Follow, Good Intro for Self Learner       | 5.0 out of 5 stars |
| 2 | Great for starting your journey on learning py... | 5.0 out of 5 stars |
| 3 | Great inner content! Not that great outer qual... | 4.0 out of 5 stars |
| 4 | Tips for beginners                                | 5.0 out of 5 stars |
| 5 | Great                                             | 5.0 out of 5 stars |
| 6 | Can be used both as a reference and a teaching... | 5.0 out of 5 stars |
| 7 | I have a whole bookshelf on Python books. Hand... | 5.0 out of 5 stars |
| 8 | simple y con el detalle de aprendizaje adecuado   | 5.0 out of 5 stars |
| 9 | Very detailed and clearly explained               | 5.0 out of 5 stars |

# Learning Python in the AI era: what really matters?

You are learning to solve problems with code — not to become a human compiler.

## 1 Learn the syntax

Understand variables, data types, conditions, loops, functions, imports and how Python code is structured.

## 2 Understand what the code does

You should be able to read code, explain its logic, change it, run it, and recognize when the result does not make sense.

## 3 Use AI — but stay in control

Vibe coding can help you generate and refine code quickly. Describe the task, test the code, inspect the output and iterate.

**You will not be evaluated on memorizing or writing long programs from scratch.**

Why?

In modern scientific work, code can be assisted by AI and reused from libraries.

What matters is whether you understand the syntax and logic well enough to ask for, inspect, adapt, debug and validate the code.

*Goal: confident, responsible problem-solvers*

# Python Learning Resources

## Think Python, 3rd Edition

*Best for: Python programming from first principles*

**Open resource:** <https://allendowney.github.io/ThinkPython/>

**Short description:** -An introductory Python book built as Jupyter notebooks;  
-Combines explanations, runnable code, exercises, and Colab use.

### How to use:

Open the site → Scroll down to ‘The notebooks’ → click the Colab link → run and edit cells.

### Study in order:

variables, functions, loops, strings, lists, dictionaries, files, classes.

# ML Foundations Resources

## ML Foundations Notebooks

*Best for: mathematical foundations of machine learning*

**Open resource:** [github.com/jonkrohn/ML-foundations/tree/master/notebooks](https://github.com/jonkrohn/ML-foundations/tree/master/notebooks)

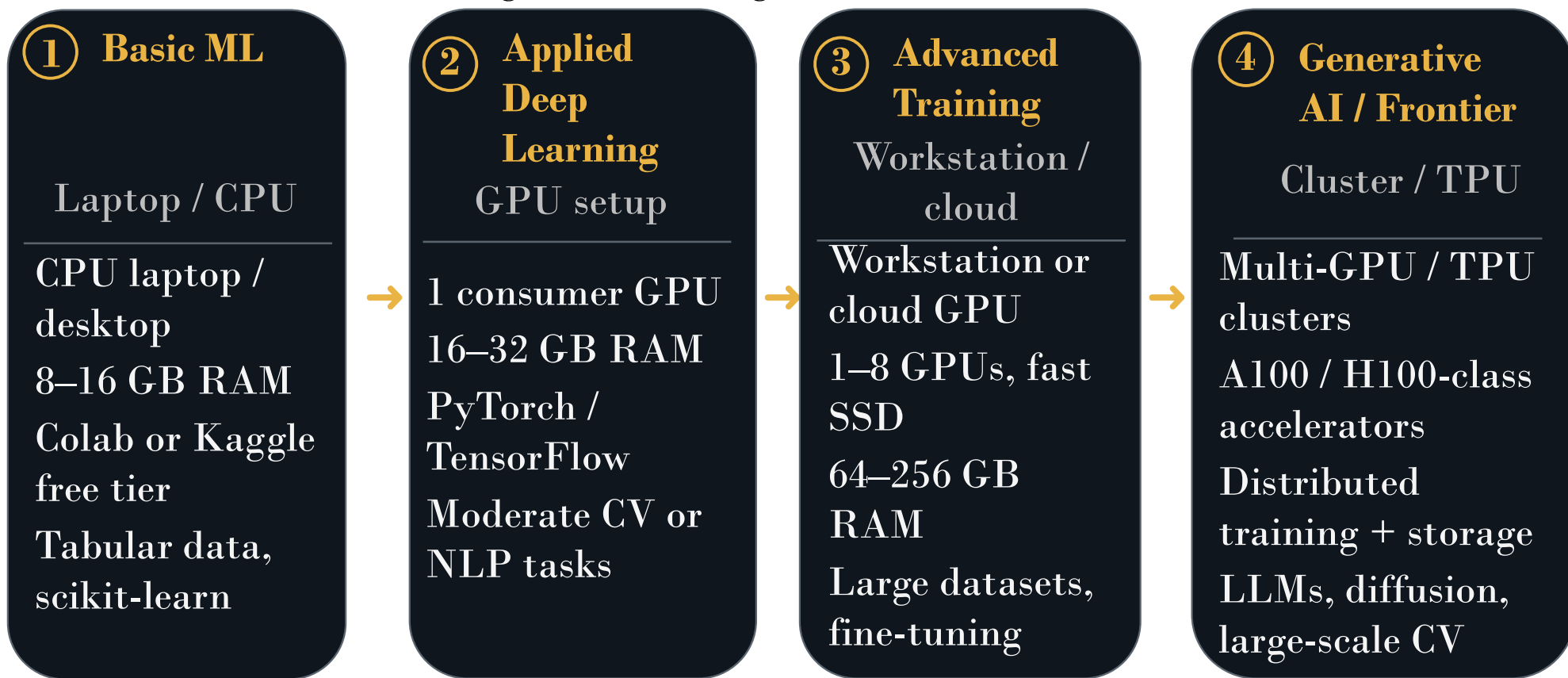
**Short description:** Hands-on notebooks for the core math behind ML: linear algebra, calculus, probability, statistics, algorithms and optimization.

### How to use:

- Open the GitHub notebook folder → choose a .ipynb file → use Open in Colab or copy the GitHub notebook URL into Colab.
- Follow the numbered notebooks in sequence.

## Compute resources for ML / AI projects

From basic machine learning to advanced generative AI



**i** | Choose the smallest setup that fits your data size, model size, and project deadline.

## Data resources for ML / AI

Widely used repositories and discovery platforms



### UCI ML Repository

Classic benchmark datasets for tabular machine learning.



### Kaggle Datasets

Community datasets, notebooks, and competitions across many domains.



### Hugging Face Datasets

Ready-to-load NLP, speech, and multimodal datasets for Python workflows.



### OpenML

Reproducible ML datasets, tasks, and experiment benchmarks.



### Google Dataset Search

Useful for discovering datasets hosted across the web.



### World Bank Open Data

Country-level indicators for health, economics, education, and development.



Choose data based on relevance, licensing, documentation, and data quality.

## Data resources for ML / AI

Useful repositories for African research and applications

### 1 Zindi

African ML competition platform with real-world datasets and challenges.

### 2 DataFirst

Survey, census, and socioeconomic microdata focused on Africa.

### 3 Afrobarometer

Public opinion survey data from many African countries.

### 4 DHS / IPUMS-DHS

Health and demographic survey data widely used across Africa.

### 5 OpenAfrica

Open civic and development datasets curated for African contexts.

### 6 Masakhane resources

Datasets and benchmarks supporting NLP for African languages.



Combine local data with global sources, and always check ethics, consent, and representativeness.



# Git, GitHub, and GitHub Codespaces

GitHub is a web-based platform that uses Git to store and manage code in repositories.

## 1 Git

### Version control tool

- Tracks changes to files and project history
- Supports commits, branches, merging, and rollback
- Works locally on your computer
- Can be used without GitHub or internet access

## 2

## GitHub

### Code hosting & collaboration platform

- Hosts Git repositories online
- Supports code sharing, teamwork, and backup
- Provides pull requests, issues, and project management
- Adds collaboration features on top of Git

## 3

## GitHub Code Spaces

### Cloud development environment

- Provides an online coding workspace
- Includes editor, terminal, compute, and installed tools
- Useful for quick setup and working from anywhere
- Uses Git/GitHub repositories inside the environment

In short: Git manages versions, GitHub helps share and collaborate, and Codespaces provides the cloud computer for writing and running the code.

**Git = version history**

**GitHub = sharing + collaboration**

**Codespaces = online dev environment**

## Why do we need it over Google Drive?

### GitHub

For code & computational projects

### Google Drive

For general file storage & documents

#### Purpose

Develop and manage code

Store and share files

#### Change history

Commits show what changed and why

File versions, less code-focused

#### Collaboration

Branches, pull requests, code review

Sharing and co-editing documents

#### Reproducibility

Code + notebooks + README together

Can store them, but no repository workflow

#### Portfolio

Repositories can showcase project work

Primarily private/general file organization

Drive stores the files. GitHub helps us manage the evolving code behind the work.  
Think of GitHub as: project history + backup + sharing + collaboration + evidence of your work.

# Getting Started with GitHub

<https://docs.github.com/en/account-and-profile/how-tos/account-management/creating-an-account-on-github>



GitHub  
<https://github.com>

## 1 Sign up

Go to GitHub and choose Sign up. You can create a personal account and verify your email address.

## 2 Choose your identity

Select a sensible username. Your username becomes part of your GitHub identity and appears in repository addresses.

## 3 Secure the account

Use a strong unique password and enable two-factor authentication (2FA). Keep recovery information safe.

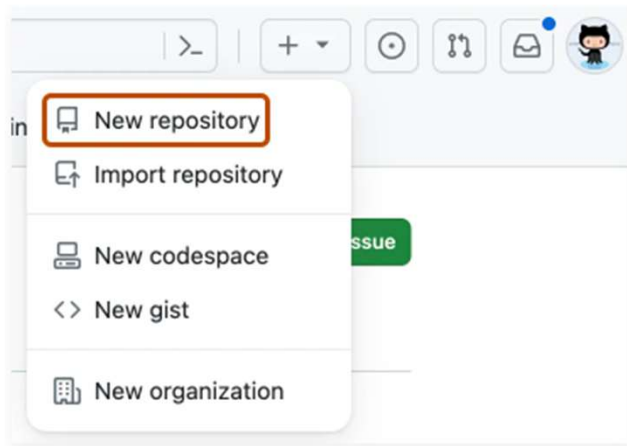
**Tip: use a name you can still use professionally after this course.**

<https://docs.github.com/en/pages>

# Create Your (First) Repository

A repository is the online project space that stores files and their change history.

1 In the upper-right corner of any page, select +, then click **New repository**.



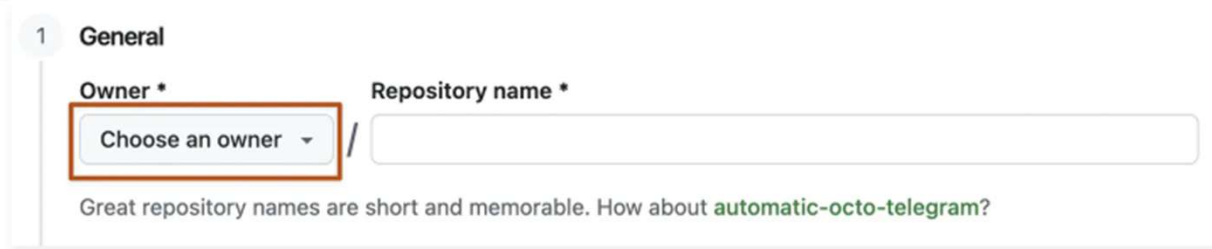
3 Type a name for your repository and an optional description.

4 Choose a repository visibility.

5 Toggle **Add README** to On.

6 Click **Create repository**.

2 Use the **Owner** dropdown menu to select the account you want to own the repository.

A screenshot of the 'General' tab in the GitHub repository creation form. The form has a light gray background. At the top left, there is a tab labeled '1 General'. Below it, there are two main sections: 'Owner \*' and 'Repository name \*'. The 'Owner \*' section contains a dropdown menu with the text 'Choose an owner' and a downward arrow. This dropdown menu is highlighted with a red rectangular box. To the right of the dropdown is a text input field for the repository name. Below these fields, there is a line of text: 'Great repository names are short and memorable. How about automatic-octo-telegram?'. The 'Repository name \*' field is empty.

**Repository = one project. Avoid putting every unrelated notebook you own into one repository.**

# Working with files in a GitHub repository

Three simple ways to get files into a repository:

## 1 Create a file directly in GitHub

Use GitHub's editor to create a new text-based file.

**Add file → Create new file**

Examples:

.md, .py, .txt, .csv, .json

## 2 Upload an existing file

Add a file that already exists on your computer.

**Add file → Upload files**

Examples:

.csv, .docx, .pdf, .png, .ipynb

## 3 Save a Colab notebook to GitHub

Send the notebook you are working on directly from Colab.

**File → Save a copy in GitHub**

The notebook is saved as an .ipynb file

**Start simple: create, upload, or save from Colab. Git commands can come later.**

## **Week 2 Assignment Task: Python and Computing Environment Setup**

- Create a new Google Colab notebook and give it an appropriate name.
- Add a Markdown cell containing the assignment title.
- Write a small Python program that:
  - asks the user to enter a temperature in °C and converts it to °F;
  - reports whether the temperature is above or below a specified threshold;
  - uses a loop to convert several temperatures; and - uses a function to perform the temperature conversion.
- Add brief comments explaining what your code does.
- Save the completed notebook in your course-project GitHub repository.
- Submit the GitHub link to your notebook on Google Classroom.